Since 1983

MIT-WPU

॥ विश्वशान्तिर्गुरुर्वं शुवा ॥

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

UDP-Based Client–Server Communication

Mini-project presentation

Computer Networks Laboratory (CSE20110)

Department of Computer Engineering and Technology

1262241930 Mitansh Dekate

1262243385 Om Mali

1262243181 Kartik Saxena

1262241292 Harpal Kalankar

Group [11] Panel [k (iOS)] Batch [2 & 3]

Faculty: [Dr.Uma pujeri Mam,Dr. Swati Mam]

Academic Year: 2026–2027

Presentation date: [10/26/2026]

Problem Statement

Communication between two applications requires a mechanism to transmit and receive data over a network. This project demonstrates message exchange between a client and a server using the User Datagram Protocol (UDP).

Objectives

1. Establish UDP communication between a client and a server.
2. Send and receive text messages using Java networking classes.
3. Understand IP addressing, port numbers, datagrams, and connectionless communication.

System Architecture

Client Application → UDP Datagram → Server Application

Client Application ← UDP Datagram ← Server Application

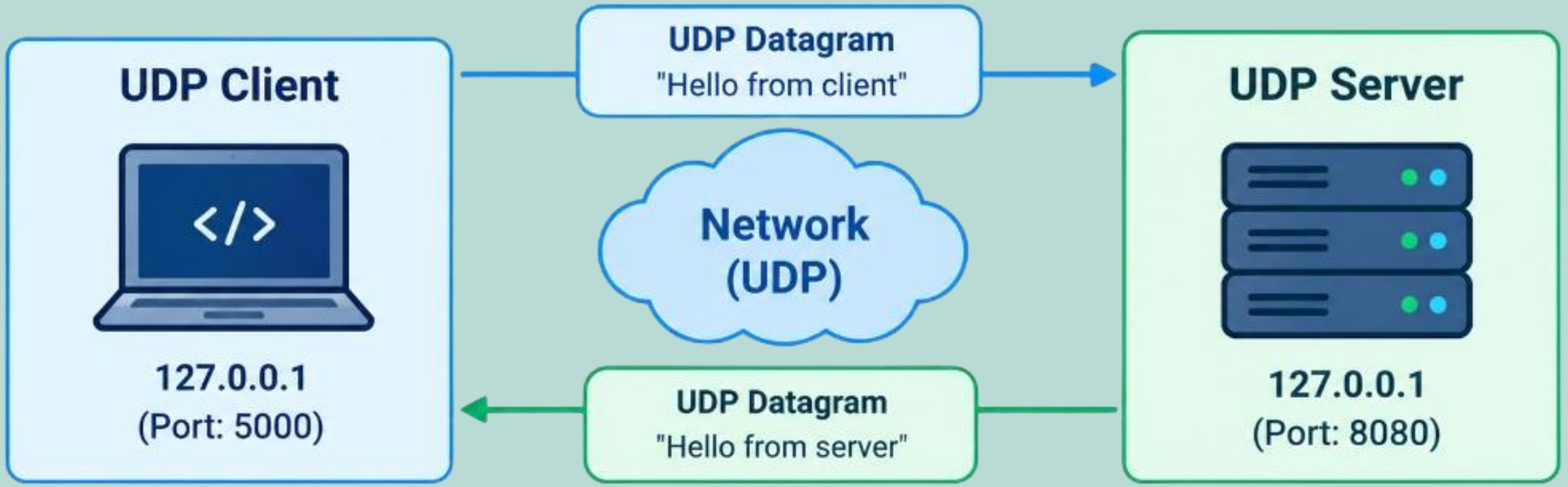
Tool

1. Programming Language: Java 8
2. Networking Classes: DatagramSocket and DatagramPacket
3. Environment: JDK and terminal/IDE

Protocol and Addressing

- Protocol: User Datagram Protocol (UDP)
- Server Port: 8080
- Server IP: 127.0.0.1 (localhost for local testing)
- Socket Type: Connectionless datagram communication

Both applications run on the same computer during localhost testing.



Implementation

1. The server creates a DatagramSocket on port 8080.
2. The client creates a UDP datagram containing "Hello from client".
3. The server receives the datagram using DatagramPacket.
4. The server sends "Hello from server" back to the client's address and port.
5. The client receives and displays the server's reply.

Working

1. Start the server application.
2. Run the client application.
3. Client sends the request using UDP.
4. Server receives and processes the message.
5. Server sends a reply, which the client displays.

Troubleshooting

A port conflict or incorrect destination port can prevent successful communication. Ensure that the server and client use the correct port and that the server starts first.

Test Case: UDP Message Exchange

Input: "Hello from client"

Expected Result:

1. Server receives the client's message.
2. Server sends "Hello from server".
3. Client receives and displays the reply.

Objectives Achieved:

- UDP sockets created successfully.
- Client-to-server message transmission tested.
- Server-to-client response tested.

```
== 1. startup (vote, 5 nodes, round 1)
  PASS  all nodes agree on one ready leader  [leader = node 3]

== 2. normal chat with two clients on different nodes
  PASS  both clients receive all 10 messages  [alice=10 bob=10]
  PASS  identical total order on both clients
  PASS  sequence numbers are gapless

== 3. crash the leader (node 3) while clients keep typing
  PASS  a new leader is elected after the crash  [node 2, 2.6s after kill]
  PASS  no message lost across the failover (20 total on both clients)  [alice=20 bob=20]
  PASS  no duplicates
  PASS  same order on both clients
  PASS  every live node holds the same committed log

== 4. restart node 3 (empty log) - it must rejoin and recover everything
  PASS  22 messages committed on every node, identical logs
  PASS  one leader again  [node 2]
  PASS  clients received all 22

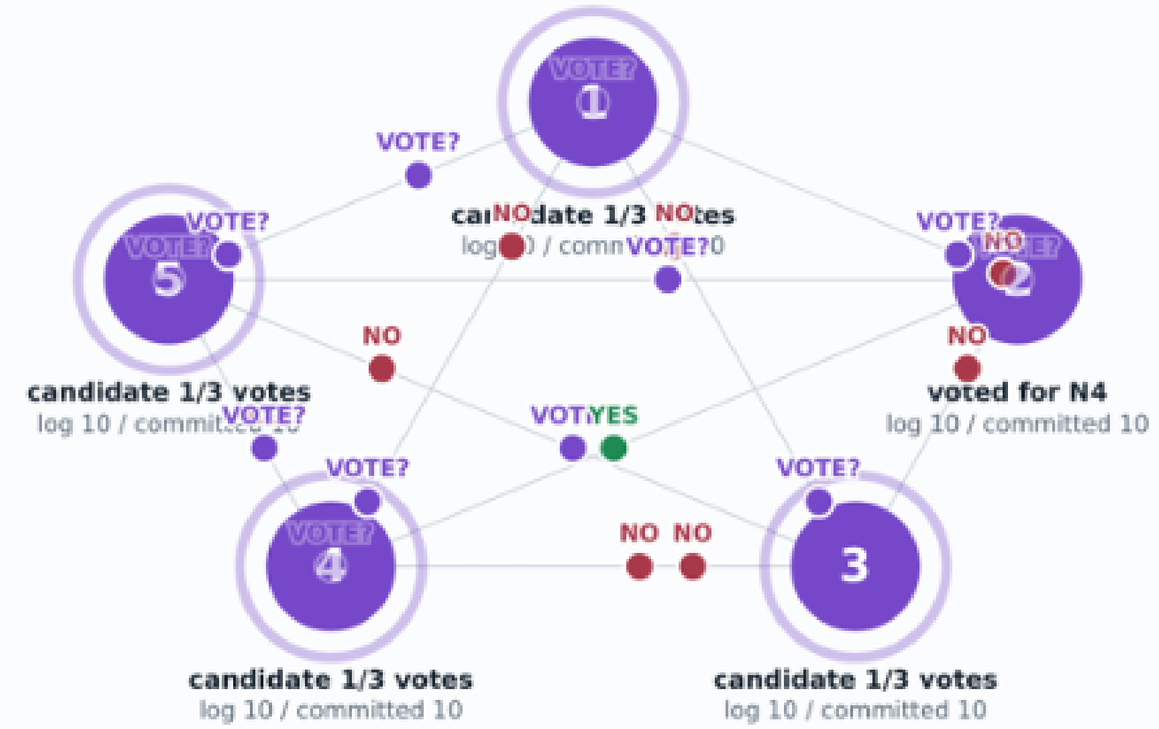
== 5. network partition of the current leader
  PASS  majority side elects a different leader  [node 3]
  PASS  majority keeps committing during the partition
  PASS  after healing, identical logs everywhere
  PASS  one leader after healing
  PASS  clients still see a gapless, duplicate-free stream  [last_seq=25]

ALL CHECKS PASSED
```

Distributed chat cluster 5 nodes | vote election | refreshed 10:16:00.352

Election in progress | 5 of 5 nodes up, need 3 to commit

Cluster click a node to select it



Failure injection

Select a node Crash node | Restart node | Cut network | Restore network

Latency none | Packet loss none | Apply to node

Whole cluster Force re-election | Crash current leader | Election speed slow motion

Election live every vote request and vote as it happens

Election in progress

Candidate N1 asking for votes in term 2

1 yes, 0 no. A majority needs 3 yes votes.

N1 ✓ itself | N2 … waiting | N3 … waiting | N4 … waiting | N5 … waiting

Candidate N3 asking for votes in term 2

1 yes, 0 no. A majority needs 3 yes votes.

N1 … waiting | N2 … waiting | N3 ✓ itself | N4 … waiting | N5 … waiting

Candidate N4 asking for votes in term 2

1 yes, 0 no. A majority needs 3 yes votes.

N1 … waiting | N2 … waiting | N3 … waiting | N4 ✓ itself | N5 … waiting

Candidate N5 asking for votes in term 2

1 yes, 0 no. A majority needs 3 yes votes.

N1 … waiting | N2 … waiting | N3 … waiting | N4 … waiting | N5 ✓ itself

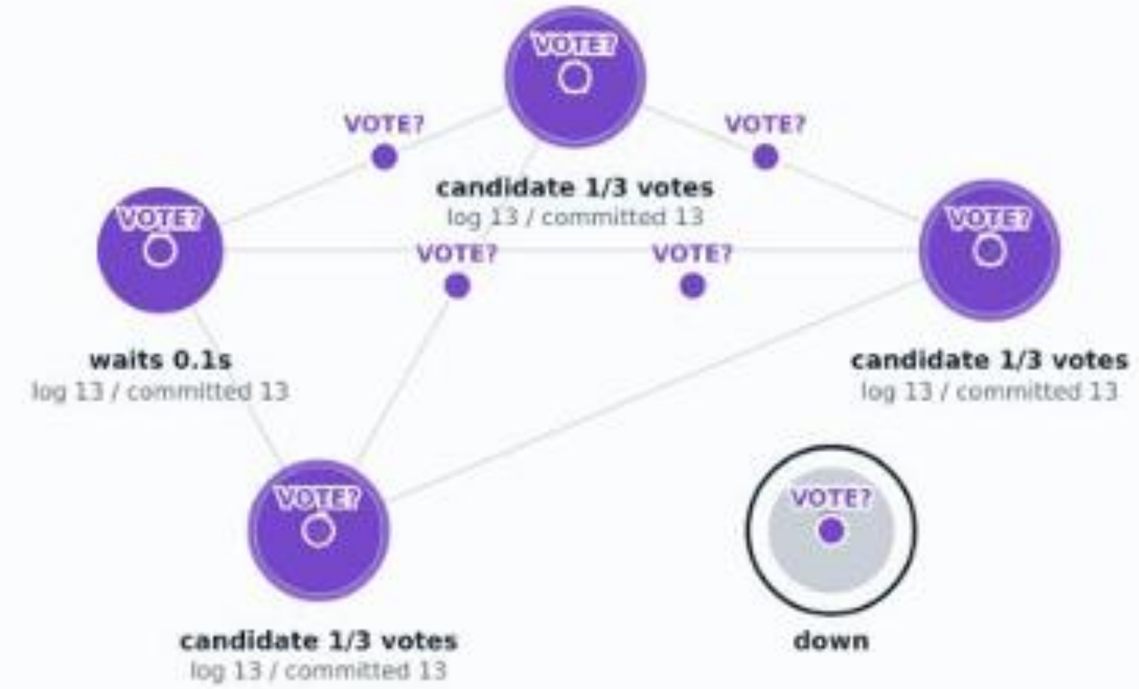
N2 voted for **N4** in term 2 and waits for the result.

Last result: N1 won the election for term 1 with 3 of 5 votes (10:15:42.710)

Distributed chat cluster 5 nodes | vote election | refreshed 10:16:16.418

Election in progress | 4 of 5 nodes up, need 3 to commit

Cluster click a node to select it



Failure injection

Node 3 Crash node | **Restart node** | Cut network | Restore network

Latency none | Packet loss none | Apply to node

Whole cluster Force re-election | Crash current leader | Election speed slow motion

Election live every vote request and vote as it happens

Election in progress

N5 has no leader. It drew a random timeout and becomes a candidate in **0.1 s** unless someone asks for its vote first.

Candidate N1 asking for votes in term 4

1 yes, 0 no. A majority needs 3 yes votes.

N1 ✓ itself | N2 … waiting | N3 down | N4 … waiting | N5 … waiting

Candidate N2 asking for votes in term 4

1 yes, 0 no. A majority needs 3 yes votes.

N1 … waiting | N2 ✓ itself | N3 down | N4 … waiting | N5 … waiting

Candidate N4 asking for votes in term 4

1 yes, 0 no. A majority needs 3 yes votes.

N1 … waiting | N2 … waiting | N3 down | N4 ✓ itself | N5 … waiting

Last result: N1 won the election for term 1 with 3 of 5 votes (10:15:42.710)

Chat sent through the cluster, delivered in one agreed order

#6 **web-ui** burst 2679-6 L584 via N1

#7 **web-ui** burst 2679-7 L613 via N1

#8 **web-ui** burst 2679-8 L644 via N1

#9 **web-ui** burst 2679-9 L657 via N1

#10 **web-ui** burst 2679-10 L687 via N1

#11 **web-ui** burst 2570-1 L1291 via N3

Nodes

Node	State	Follows	Term	Log	Committed	Log fingerprint	Clients	Network
N1	ELECTION	-	3	13	13	c56fbcf6	-	ok
N2	ELECTION	-	3	13	13	c56fbcf6	web-ui	ok
N3	DOWN	not responding						
N4	ELECTION	-	3	13	13	c56fbcf6	-	ok
N5	ELECTION	-	3	13	13	c56fbcf6	-	ok

All 4 running replicas hold the same 13 committed messages (fingerprints match).

web-ui | Write a message | Send | Send 10

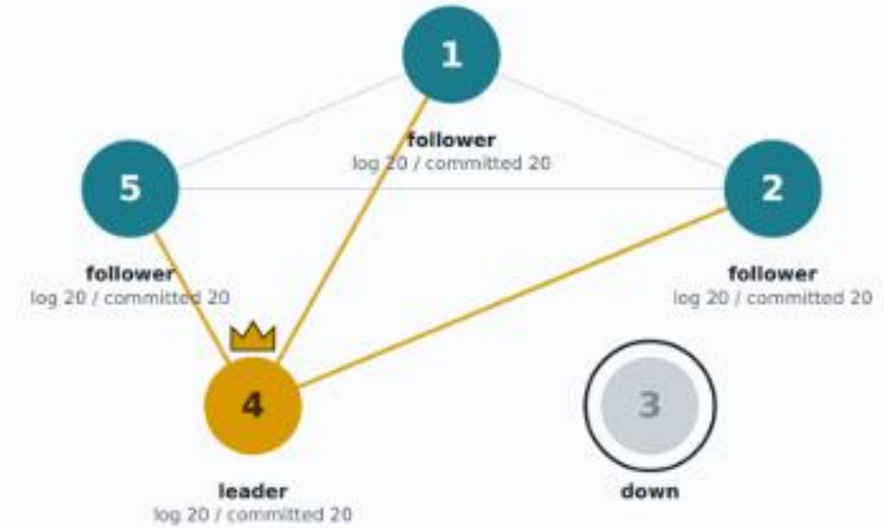
Dashboard client is attached to node 2 | last delivered #13 | waiting for delivery: 7 | connected to node 2

Events

all nodes, newest first

18:18:16.165	**N1**	timeout expired: now a CANDIDATE for term 4, voted for itself (1 of 3 votes needed)
18:18:15.816	**N2**	timeout expired: now a CANDIDATE for term 4, voted for itself (1 of 3 votes needed)
18:18:15.556	**N4**	timeout expired: now a CANDIDATE for term 4, voted for itself (1 of 3

Cluster click a node to select it



Leader · Follower · Election running · Down · Cut off from network
vote request / election message · vote YES · vote NO · new leader announced

Failure injection

Node 3 Crash node | **Restart node** | Cut network | Restore network

Latency none · Packet loss none · Apply to node

Whole cluster **Force re-election** | **Crash current leader** Election speed slow motion

Slow motion holds each election message in flight so every vote can be watched.

Nodes

Node	State	Follows	Term	Log	Committed	Log fingerprint	Clients	Network
N1	FOLLOWER	N4	5	20	20	c7528d7	-	ok
N2	FOLLOWER	N4	5	20	20	c7528d7	-	ok
N3	DOWN	not responding						
N4	LEADER	N4	5	20	20	c7528d7	web-ui	ok
N5	FOLLOWER	N4	5	20	20	c7528d7	-	ok

All 4 running replicas hold the same 20 committed messages (fingerprints match).

Election live every vote request and vote as it happens

No election running. Node 4 leads term 5.

Last result: N4 won the election for term 5 with 3 of 5 votes (10:16:22.331)

Chat sent through the cluster, delivered in one agreed order

#13	**web-ui** burst 2570-3	L1337 via N3
#14	**web-ui** burst 2570-4	L1638 via N4
#15	**web-ui** burst 2570-5	L1643 via N4
#16	**web-ui** burst 2570-6	L1648 via N4
#17	**web-ui** burst 2570-7	L1653 via N4
#18	**web-ui** burst 2570-8	L1658 via N4
#19	**web-ui** burst 2570-9	L1663 via N4
#20	**web-ui** burst 2570-10	L1668 via N4

web-ui | Write a message | **Send** | **Send 10**

Dashboard client is attached to node 4 | last delivered #20 | waiting for delivery: 0 | connected to node 4

Events all nodes, newest first

10:16:22.333	**N1**	accepted N4 as LEADER (term 5)
10:16:22.353	**N2**	accepted N4 as LEADER (term 5)
10:16:22.353	**N5**	accepted N4 as LEADER (term 5)
10:16:22.332	**N4**	leader ready: log=13 commit=13 (history synced from peers)
10:16:22.331	**N4**	became LEADER (term 5) - collecting peer logs before serving
10:16:22.331	**N4**	WON the election for term 5 with 3 of 5 votes
10:16:22.331	**N4**	received vote from N1: YES - tally 3 of 3 needed
10:16:22.229	**N4**	received vote from N5: YES - tally 2 of 3 needed
10:16:22.103	**N4**	received vote from N2: NO (already voted in term 5) - tally 1 of 3 needed
10:16:21.919	**N5**	voted NO for N2 (term 5): already voted in term 5
10:16:21.874	**N1**	voted NO for N2 (term 5): already voted in term 5

Election cost messages sent by all nodes since start

vote requests: **40** votes: **42** ELECTION: **0** OK: **0** ring tokens: **0** COORDINATOR: **0**
all messages: **2031**

When	Winner	Algorithm	Term	Election took
10:16:22.331	N4	vote	5	3.45 s
10:15:42.710	N1	vote	1	2.63 s

CONCLUSION

The project demonstrates two-way message exchange between a UDP client and server using Java networking classes.
Application: Basic network communication between applications.

Limitation: UDP does not guarantee message delivery, ordering, or retransmission.

Future Scope: Extend the application to exchange user-entered messages repeatedly and support communication between two computers on a network.

Team Contributions:

- Mitansh Dekate: Problem statement, objectives, and UDP introduction.
- Om Mali: Java implementation, client-server communication, and demonstration.
- Kartik Saxena: System architecture, tools, protocol, and addressing.
- Harpal Kalankar: Testing, results, limitations, and conclusion.

AI Use: [AI Use: Minimal AI assistance was used for documentation only.]

References

[1] Oracle, "Class DatagramSocket," Java Platform API Documentation.
https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/DatagramSocket.html

[2] Oracle, "Class DatagramPacket," Java Platform API Documentation.
https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/DatagramPacket.html

[3] Oracle, "Package java.net," Java Platform API Documentation.
https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/package-summary.html

Thank You